

Comparison of ABC and Python Programming Languages

Submitted by

Emmanuel Owusu Atta

Student ID: 01253308B

Course: Programming with Python

1 Introduction

24 July, 2026

ABC and Python are both high-level, interactive languages designed at *Centrum Wiskunde & Informatica* (CWI) in Amsterdam. ABC was designed between 1975 and 1985 by Leo Geurts, Lambert Meertens and Steven Pemberton as an easy-to-learn replacement for BASIC, Pascal and AWK, aimed chiefly at beginners and non-professional programmers [1, 3]. Guido van Rossum worked on the ABC implementation team at CWI in the 1980s, and when he began writing Python in December 1989 he deliberately reused many of ABC’s ideas – indentation-based grouping, high-level built-in data types, and an interactive read-eval-print style – while fixing what he considered ABC’s biggest flaw: its closed, non-extensible design [4]. This report summarises what the two languages’ documentation reveals about their design goals, and then compares five short programs written in both languages to see how those goals show up in practice.

2 Documentation-Based Comparison

Aspect	ABC	Python
Origin	CWI, Netherlands, 1975–1985	CWI-inspired, van Rossum, 1989 onward
Purpose	Teaching / prototyping tool for beginners, not for systems programming [2]	General-purpose language usable for scripting, web, data science, systems glue code
Execution model	Interactive integrated environment (editor + interpreter); no separate files or compile step	Interpreter (CPython) runs standalone .py files or a REPL
Statement grouping	Indentation (“off-side rule”) [5]	Indentation, directly inherited from ABC [4]
Data types	Exactly five built-in types: numbers, texts, compounds, lists (always sorted), tables [5]	Rich, open-ended type system: int, float, str, list, tuple, dict, set, classes, etc.
Variables	No declarations; a name’s type is fixed for the lifetime of one invocation	No declarations; a name may be rebound to any type at any time (dynamic typing)
Numbers	Arbitrary-precision exact (rational) and approximate (float) numbers, distinguished explicitly (~)	Arbitrary-precision int; separate float type; no built-in exact-rational type
Procedures	HOW TO definitions for commands, HOW TO RETURN for functions, HOW TO REPORT for predicates	Single def keyword for all functions; booleans are ordinary return values
Extensibility	Deliberately closed: no modules, no way to add libraries or types [3]	Extensive standard library plus third-party packages (pip); modules, classes, C extensions
I/O and systems access	Cannot directly access the file system or OS; GUI support never added [5]	Full file, network, OS and GUI access via standard library
Error handling	Combination of static and dynamic consistency checks; no user-level exceptions	try/except exceptions, whose syntax and semantics were influenced by Modula-3, not ABC [4]

Table 1: Summary of similarities and differences drawn from the ABC Programmer’s Handbook and Python documentation.

Both languages therefore share a philosophy of readability and a syntax built around indentation and high-level data structures, but they diverge sharply on scope: ABC is a closed teaching environment with five fixed types and no access to the operating system, while Python is an open, extensible general-purpose language with a vast ecosystem of libraries.

3 Programs Compared

Five short programs were written in both languages: a simple arithmetic calculator, a Fibonacci sequence generator, a leap-year checker, a multiplication (times) table generator, and a three-number average calculator. All Python programs were run and verified to produce correct output. ABC has no publicly runnable modern interpreter reachable from this environment, so the ABC programs were written by hand, following the syntax and built-in operations documented in the *ABC Programmer's Handbook* [1] (HOW TO, PUT ... IN, WRITE, READ ... EG, SELECT/ELSE, FOR ... IN {lo..hi}).

1. Simple Calculator

ABC

```
HOW TO CALCULATE:
WRITE "Enter first number: "
READ x EG 0
WRITE "Enter operator (+,-,*,/): "
READ op RAW
WRITE "Enter second number: "
READ y EG 0
SELECT:
  op = "+": WRITE x, "+", y, "=", x+y /
  op = "-": WRITE x, "-", y, "=", x-y /
  op = "*": WRITE x, "*", y, "=", x*y /
  op = "/": WRITE x, "/", y, "=", x/y /
ELSE: WRITE "Unknown operator" /
```

Python

```
def calculator(x, op, y):
    if op == "+":
        return x + y
    elif op == "-":
        return x - y
    elif op == "*":
        return x * y
    elif op == "/":
        return x / y
    else:
        return None

x = float(input("Enter first number: "))
op = input("Enter operator (+,-,*,/): ")
y = float(input("Enter second number: "))
result = calculator(x, op, y)
if result is None:
    print("Unknown operator")
else:
    print(f"{x} {op} {y} = {result}")
```

2. Fibonacci Sequence

ABC

```
HOW TO PRINT FIBONACCI n:
PUT 0, 1 IN a, b
FOR i IN {1..n}:
    WRITE a /
    PUT b, a+b IN a, b
```

Python

```
def fibonacci(n):
    a, b = 0, 1
    result = []
    for _ in range(n):
        result.append(a)
        a, b = b, a + b
    return result

n = int(input("How many Fibonacci numbers? "))
for value in fibonacci(n):
    print(value)
```

3. Leap Year Check

ABC

```
HOW TO CHECK LEAP year:
SELECT:
  (year mod 4 = 0 AND year mod 100 <> 0) OR year
  mod 400 = 0:
    WRITE year, "is a leap year" /
ELSE:
  WRITE year, "is not a leap year" /
```

Python

```
def is_leap(year):
    return (year % 4 == 0 and year % 100 != 0) or \
           year % 400 == 0

year = int(input("Enter a year: "))
if is_leap(year):
    print(f"{year} is a leap year")
else:
    print(f"{year} is not a leap year")
```

4. Times Table (2 to 12)

ABC

```
HOW TO PRINT TABLE n:
FOR i IN {1..10}:
    WRITE n, "x", i, "=", n*i /

HOW TO PRINT TABLES:
FOR n IN {2..12}:
    PRINT TABLE n
    WRITE /
```

Python

```
def print_table(n):
    for i in range(1, 11):
        print(f"{n} x {i} = {n * i}")

for n in range(2, 13):
    print_table(n)
    print()
```

5. Average of Three Numbers

ABC

```
HOW TO PRINT AVERAGE OF a, b, c:
PUT (a+b+c)/3 IN avg
WRITE "The average is", avg /
```

Python

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

average = (a + b + c) / 3
print(f"The average is {average}")
```

4 Findings

Similarities

- **Indentation for structure.** Neither language uses braces or BEGIN/END; a colon followed by an indented block marks the body of an IF, SELECT/if-elif-else, FOR, or WHILE. Compare ABC's `FOR i IN {1..n}`: with Python's `for i in range(n):`.
- **High-level, low-ceremony data handling.** Both let the calculator and average programs be written without any type declarations, and both support tuple-like multiple assignment: ABC's `PUT b, a+b IN a, b` in the Fibonacci program is mirrored almost exactly by Python's `a, b = b, a + b`.
- **Arbitrary-precision integers.** Both handle integers of unlimited size natively, so the Fibonacci program never overflows in either language.
- **Readable, near English-like keywords.** IF, FOR, WHILE, ELSE appear in both, and both favour few keywords over punctuation-heavy syntax.

Differences

- **Assignment direction and keywords.** ABC always uses `PUT expression IN location` (value first, destination named explicitly); Python uses ordinary `name = expression` assignment, which is shorter and closer to conventional mathematical notation.
- **Procedure vocabulary.** ABC distinguishes three kinds of how-to: `HOW TO` for commands, `HOW TO RETURN` for functions, and `HOW TO REPORT` for predicates (Boolean tests). Python collapses all of these into a single `def`, since a Python function can return any value, including `True/False`, print output as a side effect, or do both.
- **Multi-way branching.** ABC's leap-year and calculator programs use `SELECT/ELSE` with a list of independent tests; Python uses `if/elif/else` chained explicitly. The behaviour is equivalent but ABC's form reads more like a guarded case statement.
- **Ranges and loop variables.** ABC generates the times table using the literal range notation `{2..12}` over a *list*, since ABC lists are a core built-in type; Python uses the `range()` function, which produces a lazy sequence rather than a concrete list.
- **Library ecosystem and extensibility.** None of the five ABC programs could be extended with third-party code, because ABC has no import mechanism and cannot reach the file system [3]. The same Python programs could immediately be extended with modules such as `math`, `argparse`, or a GUI toolkit – something the ABC designers explicitly chose not to support.
- **Numeric model.** ABC's calculator can perform exact rational division (e.g. `1/3` stays exact until approximated), whereas Python's `/` operator always returns a `float`; Python only keeps exact results when using `fractions.Fraction` explicitly.
- **Tooling and portability.** ABC is tied to its own bespoke editor/environment and was only ever released for a handful of 1990s platforms (DOS, classic Mac OS, early Unix, Atari TOS) [2]. Python runs as an ordinary interpreter on virtually any modern operating system and integrates with standard text editors, IDEs, and build tools.

5 Conclusion

Reading the ABC Programmer's Handbook alongside Python's own documentation confirms the well-known history: Python is a direct descendant of ABC's syntax and philosophy of readability, high-level data types, and indentation-based structure, but it was built specifically to remove the constraints that limited ABC – lack of extensibility, no file/OS access, and a closed, single-implementation environment. The five short programs show this concretely: the control-flow and assignment idioms translate almost line-for-line between the two languages, while anything requiring libraries, file access, or open-ended extension is only possible in Python.

References

- [1] Geurts, L., Meertens, L., Pemberton, S. *The ABC Programmer's Handbook*, Chapter 1: A Quick Look at ABC. <https://homepages.cwi.nl/~steven/abc/programmers/quick-look.html>
- [2] *ABC (programming language)*, HandWiki, based on Wikipedia. [https://handwiki.org/wiki/ABC_\(programming_language\)](https://handwiki.org/wiki/ABC_(programming_language))
- [3] *ABC (programming language)*, Grokipedia. [https://grokipedia.com/page/ABC_\(programming_language\)](https://grokipedia.com/page/ABC_(programming_language))
- [4] *General Python FAQ*, Python 3 Documentation. <https://docs.python.org/3/faq/general.html>
- [5] *ABC Programming Language Overview and Key Features*. <https://www.studocu.com/in/document/university-of-calicut/bachelors-of-computer-application-bca/abc-programming-language/43342699>